

算法与算法复杂性理论 - 作业 5 [cite: 22, 23]

1. DGIM 算法分析 [cite: 24]

DGIM 算法用于在滑动窗口中近似计数 1 的个数，其核心是维护大小为 2 的幂的桶，且同一大小的桶最多允许 1 到 2 个，桶大小从新到旧单调不减 [cite: 24, 25, 26, 27]。

(a) 证明 DGIM 算法的空间复杂度为 $O(\log^2 N)$ [cite: 29]

证明. 对于大小为 N 的滑动窗口，窗口内最多有 N 个“1”。

根据 DGIM 算法的规则，桶的大小为 2^i ，因此最大的桶大小不超过 N 。所以桶大小的种类数最多为 $\lceil \log_2 N \rceil + 1$ 种，即 $O(\log N)$ 种。

又因为每种大小的桶最多有 2 个，所以滑动窗口中维护的桶的总数也是 $O(\log N)$ 个。

对于每个桶，我们需要记录它的时间戳和桶的大小：

- **时间戳：**只需要记录其在窗口内的相对位置即可，需要 $O(\log N)$ 个比特（以 N 为模进行运算）。
- **桶的大小：**由于大小是 2^i ，只需记录指数 i 即可，需要 $O(\log \log N)$ 个比特。

因此，每个桶需要存储的空间上限为 $O(\log N)$ 比特。

总空间复杂度 = 桶的总数 $\times$ 每个桶的空间 = $O(\log N) \times O(\log N) = O(\log^2 N)$ 比特。证明完毕。 $\square$

(b) 计算估计值与真实值的相对误差 [cite: 30, 31, 32]

已知当前窗口大小 $N = 16$ ，时刻为 t ，查询最近 $k = 7$ 位中 1 的个数。

当前桶配置为： $(t, 1), (t - 2, 1), (t - 5, 2), (t - 8, 2), (t - 12, 4)$ [cite: 31]。

查询范围为时刻 t 到 $t - 6$ 。我们需要找到时间戳在 $t - 6$ 及以后的最旧的桶 [cite: 28]。

- 桶 $(t - 5, 2)$ 的时间戳为 $t - 5 \geq t - 6$ ，在查询范围内。
- 桶 $(t - 8, 2)$ 的时间戳为 $t - 8 < t - 6$ ，不在查询范围内。

因此，最旧的桶 b 为 $(t - 5, 2)$ ，其大小为 2。

根据 DGIM 估计规则，估计值 = b 之后更近的所有桶大小之和 + b 的大小的一半 [cite: 28]。

更近的桶有 $(t, 1)$ 和 $(t - 2, 1)$ ，其大小之和为 $1 + 1 = 2$ 。

估计值 = $2 + 2/2 = 3$ 。

已知最近 7 位的真实内容为 (从新到旧): 1, 0, 1, 0, 0, 1, 1 [cite: 32]。

可以看到其中真实的 1 的个数为: **真实值** = 4。

相对误差 = $\frac{|3-4|}{4} = \frac{1}{4} = 25\%$ 。

(c) 时刻 $t+1$ 到达比特 “1” 的完整更新过程 [cite: 33]

当新时刻 $t+1$ 到达一个 “1” 时, 生成一个新的大小为 1 的桶: $(t+1, 1)$ 。

此时当前的所有桶配置如下 (从新到旧):

$(t+1, 1), (t, 1), (t-2, 1), (t-5, 2), (t-8, 2), (t-12, 4)$ 。

由于大小为 1 的桶变成了 3 个, 违反了最多 2 个的限制, 触发级联合并操作:

1. **合并最旧的两个大小为 1 的桶:** $(t, 1)$ 和 $(t-2, 1)$ 。合并后生成大小为 2 的桶, 时间戳保留较新的即 t 。

此时桶配置: $(t+1, 1), (t, 2), (t-5, 2), (t-8, 2), (t-12, 4)$ 。

2. **合并最旧的两个大小为 2 的桶:** 此时大小为 2 的桶有 $(t, 2), (t-5, 2), (t-8, 2)$ 共 3 个。合并最旧的两个 $(t-5, 2)$ 和 $(t-8, 2)$, 生成大小为 4 的桶, 时间戳保留 $t-5$ 。

此时桶配置: $(t+1, 1), (t, 2), (t-5, 4), (t-12, 4)$ 。

3. **检查大小为 4 的桶:** 现有 $(t-5, 4)$ 和 $(t-12, 4)$ 共 2 个, 满足限制, 合并结束。

最终的桶配置: $(t+1, 1), (t, 2), (t-5, 4), (t-12, 4)$ 。

(d) 证明相对误差不超过 50% [cite: 34]

证明. 设最旧且在查询范围内的桶 b 的大小为 2^j 。

设位于 b 之后 (更近) 的所有桶的大小之和为 S 。

算法的估计值为: $E = S + 2^{j-1}$ 。

真实范围内 1 的个数 c 必须包含 b 之后所有更近的桶中的 1 (即 S 个), 并且至少包含桶 b 中最新到达的那 1 个 (因为 b 的时间戳在查询范围内)。

同时, 最极端的情况下, b 桶中所有的 2^j 个 “1” 都在查询范围内。因此, 真实值 c 满足:

$$S + 1 \leq c \leq S + 2^j$$

估计值的绝对误差为 $|E - c| = |S + 2^{j-1} - c|$ 。

由于 c 的取值范围, 绝对误差的最大可能值为 2^{j-1} , 即 $|E - c| \leq 2^{j-1}$ 。

另一方面, 在 DGIM 算法中, 如果要产生大小为 2^j 的桶, 必然是由较小的桶合并而来。根据算法保持的性质 (每种大小的桶有 1 到 2 个), 在最旧的桶 b 之前必定存在大小为 $2^0, 2^1, \dots, 2^{j-1}$ 的桶各至少 1 个。

因此, $S \geq 1 + 2 + 4 + \dots + 2^{j-1} = 2^j - 1$ 。

由此推导出 $c \geq S + 1 \geq (2^j - 1) + 1 = 2^j$ 。

因此, 相对误差为:

$$\frac{|E - c|}{c} \leq \frac{2^{j-1}}{2^j} = \frac{1}{2} = 50\%$$

证明完毕。 □

2. 证明集合划分问题是 NP 完全问题 [cite: 35, 36]

证明. 我们要证明集合划分问题 (Partition Problem) 是 NP-Complete 的, 分为两步: 证明其在 NP 中, 以及证明其是 NP-Hard [cite: 36]。

第一步: 证明 Partition $\in$ NP

给定集合 S 和一个候选划分 (A, B) , 我们可以在多项式时间内验证: 1. 验证 $A \cap B = \emptyset$ 且 $A \cup B = S$ 。2. 验证 $\sum_{x \in A} x = \sum_{y \in B} y$ 。

上述过程显然可以在多项式时间内完成, 因此 Partition 问题属于 NP 类。

第二步: 证明 Partition 是 NP-Hard

我们通过从已知的 NP 完全问题 **子集和问题 (Subset Sum)** 规约来证明。

Subset Sum 问题定义为: 给定一个自然数集合 $X = \{x_1, x_2, \dots, x_n\}$ 和一个目标值 t , 判断是否存在子集 $X' \subseteq X$ 使得 $\sum X' = t$ 。

设 $W = \sum_{i=1}^n x_i$ 。我们构造一个 Partition 问题的实例 S 如下:

$$S = X \cup \{2W - t, W + t\}$$

(这里如果 X 中已存在这两个数, 可通过同时放大所有数值并加权偏移来保证元素唯一性, 为简化表述, 默认新元素不会引发多重集冲突)。

此时, 集合 S 中所有元素的总和为:

$$\sum S = W + (2W - t) + (W + t) = 4W$$

如果要将 S 划分为两个和相等的子集 A 和 B , 那么每个子集的元素和必须为 $2W$ 。

观察我们添加的两个新元素 $(2W - t)$ 和 $(W + t)$: 由于 $(2W - t) + (W + t) = 3W > 2W$, 这两个元素 **不可能** 同时存在于同一个子集 (比如 A) 中。

因此, 它们必定被分别分到 A 和 B 中。假设 $(2W - t) \in A$ 。

那么集合 A 中除了 $(2W - t)$ 以外的剩余元素必须来自原来的集合 X 。设这些元素构成子集 $X' \subseteq X$, 则:

$$\sum A = (2W - t) + \sum X' = 2W \implies \sum X' = t$$

这表明, 存在这样一个满足 Partition 条件的划分, 当且仅当在 X 中存在一个子集 X' 其和为 t 。因为我们可以多项式时间内完成从 Subset Sum 到 Partition 问题的实例规约, 并且 Subset Sum 是 NP-Hard, 所以 Partition 问题也是 NP-Hard 的。

综上, Partition 问题属于 NP 且是 NP-Hard, 因此是 NP 完全问题。 □

3. 证明 NP 中语言的判定算法运行时间 [cite: 37]

(注: 原题目中 “ $20()$ ” 疑似为 OCR 识别错误, 合理推断此题要求证明存在 $2^{O(n^k)}$ 的判定算法, 其中 k 为常数 [cite: 37])

证明. 根据 NP 的定义, 如果一个语言 $L \in \text{NP}$, 那么存在一个多项式时间的验证机 (Verifier) V 和一个多项式 $p(n)$, 使得对于任意输入字符串 x , $x \in L$ 当且仅当存在一个长度 $|y| \leq p(|x|)$ 的证书 (Certificate) y , 满足 $V(x, y)$ 接受。

由于验证机 V 运行在多项式时间内, 设其时间复杂度为 $O((|x| + |y|)^c)$, 其中 c 为常数。由于 $|y| \leq p(|x|)$, 验证给定的一对 (x, y) 的运行时间可被限界为 $O(|x|^k)$, 其中 k 也是某个确定的常数。

对于给定的输入 x (设 $|x| = n$), 我们可以设计一个确定的穷举算法来判定 x 是否属于 L :

1. 穷举所有可能的长度不超过 $p(n)$ 的二进制字符串 y 。
2. 这样的字符串共有 $\sum_{i=0}^{p(n)} 2^i = 2^{p(n)+1} - 1$ 个, 其数量级为 $O(2^{p(n)})$ 。
3. 对每一个字符串 y , 运行多项式时间验证机 $V(x, y)$ 。

该算法的总运行时间为穷举的数量乘以单次验证的时间, 即:

$$\text{Total Time} = O\left(2^{p(n)} \cdot n^k\right)$$

由于 $p(n)$ 是关于 n 的多项式, n^k 相较于指数项增长极慢, 总时间复杂度可以表示为 $2^{O(n^d)}$, 其中 d 为多项式 $p(n)$ 的最高次幂, 是一个常数。

因此, NP 中的任何语言都可以用一个运行时间为 $2^{O(n^k)}$ (指数级别) 的确定性算法来判定, 证明完毕。□